

```

"""
Smart Housing Price Analytics – Live Dashboard
Infotact Solutions Internship – Project 3
Geospatial Real Estate Valuation via Spatial Embeddings / Attention

Author: Megha K A
"""

import json
import os

import joblib
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

# -----
# PAGE CONFIG
# -----
st.set_page_config(
    page_title="Smart Housing Price Analytics",
    page_icon="🏠",
    layout="wide",
)

# -----
# CUSTOM DARK THEME / STYLING
# -----
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;

html, body, [class*="css"] {
    font-family: 'Inter', sans-serif;
}

/* App background */
.stApp {
    background: linear-gradient(180deg, #0b1120 0%, #0f172a 100%);
    color: #e2e8f0;
}

```

```

/* Headings */
h1, h2, h3 {
    font-family: 'Inter', sans-serif;
    font-weight: 800 !important;
    letter-spacing: -0.02em;
}
h1 { color: #f8fafc !important; }
h2, h3 { color: #f1f5f9 !important; }

/* Caption / subtitle text under title */
[data-testid="stCaptionContainer"] {
    color: #94a3b8 !important;
    font-size: 0.95rem !important;
}

/* Tabs */
button[data-baseweb="tab"] {
    font-weight: 600;
    color: #94a3b8;
}
button[data-baseweb="tab"][aria-selected="true"] {
    color: #22d3ee !important;
    border-bottom-color: #22d3ee !important;
}

/* KPI-style metric cards */
[data-testid="stMetric"] {
    background: linear-gradient(145deg, #111827, #1e293b);
    border: 1px solid #27364a;
    border-radius: 14px;
    padding: 18px 20px 12px 20px;
    box-shadow: 0 4px 14px rgba(0,0,0,0.25);
}
[data-testid="stMetricLabel"] { color: #94a3b8 !important; }
[data-testid="stMetricValue"] { color: #22d3ee !important; font-weight: 800 !impo

/* Buttons */
.stButton > button, .stFormSubmitButton > button {
    background: linear-gradient(135deg, #06b6d4, #0891b2) !important;
    color: white !important;
    border: none !important;
    border-radius: 10px !important;
    font-weight: 700 !important;
    padding: 0.6rem 1.4rem !important;
    box-shadow: 0 4px 12px rgba(6,182,212,0.35);
    transition: transform 0.15s ease;
}

```

```
.stButton > button:hover, .stFormSubmitButton > button:hover {
  transform: translateY(-1px);
  box-shadow: 0 6px 16px rgba(6,182,212,0.5);
}
```

```
/* Success / info / warning boxes */
[data-testid="stAlert"] {
  border-radius: 12px;
  border-left: 4px solid #22d3ee;
}
```

```
/* DataFrames */
[data-testid="stDataFrame"] {
  border-radius: 12px;
  overflow: hidden;
  border: 1px solid #27364a;
}
```

```
/* Section divider spacing */
hr { border-color: #27364a !important; }
```

```
/* Sidebar-less form container feel */
[data-testid="stForm"] {
  background: rgba(255,255,255,0.02);
  border: 1px solid #27364a;
  border-radius: 16px;
  padding: 1.2rem 1.4rem;
}
```

```
/* Kicker label above title */
.kicker {
  color: #22d3ee;
  font-weight: 700;
  font-size: 0.8rem;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  margin-bottom: 0.2rem;
}
```

```
</style>
```

```
""", unsafe_allow_html=True)
```

```
SEATTLE_LAT, SEATTLE_LON = 47.6062, -122.3321
```

```
CURRENT_YEAR = 2026
```

```
FEATURE_COLS = [
    "bedrooms", "bathrooms", "sqft_living", "sqft_lot", "floors", "waterfront",
    "view", "condition", "grade", "sqft_above", "sqft_basement", "house_age",
```

```

        "is_renovated", "years_since_renovation", "dist_to_center_km",
        "living_lot_ratio", "basement_flag", "lat", "long",
    ]

# Fallback metrics – used only if the metrics json files aren't found in the repo
FALLBACK_METRICS = {
    "Linear Regression": {"MAPE": 23.5, "RMSE": 195000, "R2": 0.70},
    "XGBoost":           {"MAPE": 12.14, "RMSE": 118000, "R2": 0.907},
    "Attention Spatial Model": {"MAPE": 13.4, "RMSE": 124000, "R2": 0.895},
}

PLOTLY_DARK = dict(
    template="plotly_dark",
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",
    font=dict(family="Inter, sans-serif", color="#e2e8f0"),
)

# -----
# HELPERS
# -----
def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    return 2 * R * np.arcsin(np.sqrt(a))

@st.cache_resource
def load_model():
    path = "models/xgboost_baseline.pkl"
    if os.path.exists(path):
        return joblib.load(path)
    return None

@st.cache_data
def load_metrics():
    result = {}
    try:
        with open("dataset/baseline_metrics.json") as f:
            b = json.load(f)
            result["Linear Regression"] = b["linear_regression"]
            result["XGBoost"] = b["xgboost"]

```

```

except (FileNotFoundError, KeyError):
    result["Linear Regression"] = FALLBACK_METRICS["Linear Regression"]
    result["XGBoost"] = FALLBACK_METRICS["XGBoost"]

try:
    with open("dataset/attention_model_metrics.json") as f:
        result["Attention Spatial Model"] = json.load(f)
except FileNotFoundError:
    result["Attention Spatial Model"] = FALLBACK_METRICS["Attention Spatial M

return result

@st.cache_data
def load_sample_listings():
    path = "dataset/processed/sample_listings.csv"
    if os.path.exists(path):
        return pd.read_csv(path)
    return None

def build_feature_row(inputs: dict) -> pd.DataFrame:
    house_age = CURRENT_YEAR - inputs["yr_built"]
    is_renovated = int(inputs["yr_renovated"] > 0)
    years_since_renovation = (
        CURRENT_YEAR - inputs["yr_renovated"] if is_renovated else house_age
    )
    dist_to_center_km = haversine(
        inputs["lat"], inputs["long"], SEATTLE_LAT, SEATTLE_LON
    )
    living_lot_ratio = inputs["sqft_living"] / max(inputs["sqft_lot"], 1)
    basement_flag = int(inputs["sqft_basement"] > 0)

    row = {
        "bedrooms": inputs["bedrooms"],
        "bathrooms": inputs["bathrooms"],
        "sqft_living": inputs["sqft_living"],
        "sqft_lot": inputs["sqft_lot"],
        "floors": inputs["floors"],
        "waterfront": inputs["waterfront"],
        "view": inputs["view"],
        "condition": inputs["condition"],
        "grade": inputs["grade"],
        "sqft_above": inputs["sqft_above"],
        "sqft_basement": inputs["sqft_basement"],
        "house_age": house_age,
        "is_renovated": is_renovated,
    }

```

```

        "years_since_renovation": years_since_renovation,
        "dist_to_center_km": dist_to_center_km,
        "living_lot_ratio": living_lot_ratio,
        "basement_flag": basement_flag,
        "lat": inputs["lat"],
        "long": inputs["long"],
    }
    return pd.DataFrame([row])[FEATURE_COLS]

def nearest_comps(df: pd.DataFrame, lat: float, long: float, k: int = 5) -> pd.DataFrame:
    d = df.copy()
    d["dist_km"] = haversine(lat, long, d["lat"].values, d["long"].values)
    return d.sort_values("dist_km").head(k)

# -----
# HEADER
# -----
st.markdown('<div class="kicker">Infotact Solutions · Project 3</div>', unsafe_allow_html=True)
st.title("🏠 Smart Housing Price Analytics")
st.caption(
    "Geospatial Real Estate Valuation via Spatial Embeddings | King County Housing Data"
)

model = load_model()
metrics = load_metrics()
sample_df = load_sample_listings()

if model is None:
    st.warning(
        "⚠️ `models/xgboost_baseline.pkl` was not found in this deployment. "
        "The **Predict Price** tab needs it – see the README for how to add it. "
        "The rest of the dashboard still works."
    )

tab1, tab2, tab3, tab4 = st.tabs(
    [
        "💰 Predict Price",
        "🇮🇹 Model Comparison",
        "🌐 Spatial Insights",
        "ℹ️ About"
    ]
)

# -----
# TAB 1 – PREDICT PRICE
# -----
with tab1:
    st.subheader("Estimate a Property's Value")
    st.write("Enter property details below – same feature set used to train the model")

```

```

with st.form("predict_form"):
    c1, c2, c3 = st.columns(3)
    with c1:
        bedrooms = st.number_input("Bedrooms", 0, 15, 3)
        bathrooms = st.number_input("Bathrooms", 0.0, 10.0, 2.0, step=0.25)
        sqft_living = st.number_input("Living area (sqft)", 200, 15000, 1800)
        sqft_lot = st.number_input("Lot size (sqft)", 300, 200000, 5000)
        floors = st.number_input("Floors", 1.0, 4.0, 1.0, step=0.5)
        waterfront = st.selectbox("Waterfront property?", [0, 1], format_func=
    with c2:
        view = st.slider("View quality (0=none, 4=excellent)", 0, 4, 0)
        condition = st.slider("Condition (1-5)", 1, 5, 3)
        grade = st.slider("Construction grade (1-13)", 1, 13, 7)
        sqft_above = st.number_input("Sqft above ground", 200, 12000, 1500)
        sqft_basement = st.number_input("Sqft basement", 0, 5000, 300)
    with c3:
        yr_built = st.number_input("Year built", 1900, CURRENT_YEAR, 1990)
        yr_renovated = st.number_input("Year renovated (0 = never)", 0, CURRE
        lat = st.number_input("Latitude", 47.0, 48.0, 47.55, format="%.4f")
        long = st.number_input("Longitude", -123.0, -121.0, -122.20, format="

    submitted = st.form_submit_button("Predict Price", type="primary")

if submitted:
    if model is None:
        st.error("Model file not available – cannot generate a prediction in
    else:
        inputs = dict(
            bedrooms=bedrooms, bathrooms=bathrooms, sqft_living=sqft_living,
            sqft_lot=sqft_lot, floors=floors, waterfront=waterfront, view=vie
            condition=condition, grade=grade, sqft_above=sqft_above,
            sqft_basement=sqft_basement, yr_built=yr_built,
            yr_renovated=yr_renovated, lat=lat, long=long,
        )
        X = build_feature_row(inputs)
        pred_price = model.predict(X)[0]

        st.success("Prediction complete")
        st.metric("Estimated Market Value", f"${pred_price:,.0f}")

    if sample_df is not None:
        st.markdown("***Top 5 nearby comparable listings**")
        comps = nearest_comps(sample_df, lat, long, k=5)
        show_cols = [c for c in ["price", "bedrooms", "bathrooms", "sqft_
        st.dataframe(
            comps[show_cols].style.format({"price": "${:,.0f}", "dist_km"
            use_container_width=True,

```

```

    )
else:
    st.info(
        "Add `dataset/processed/sample_listings.csv` to your repo to
        "comparable sales here (mirrors the Real Estate Appraiser wor
    )

# -----
# TAB 2 – MODEL COMPARISON
# -----

with tab2:
    st.subheader("Linear Regression vs. XGBoost vs. Attention Spatial Model")

    comp_df = pd.DataFrame([{"Model": m, **v} for m, v in metrics.items()])
    comp_df = comp_df.rename(columns={"MAPE": "MAPE (%)", "RMSE": "RMSE ($)", "R2":
                                     "R²"})

    k1, k2, k3 = st.columns(3)
    best_row = comp_df.loc[comp_df["MAPE (%)"].idxmin()]
    k1.metric("Best Model", best_row["Model"])
    k2.metric("Lowest MAPE", f"{best_row['MAPE (%)']:.2f}%")
    k3.metric("Best R²", f"{best_row['R²']:.3f}")

    st.dataframe(
        comp_df.style.format({"MAPE (%)": "{:.2f}", "RMSE ($)": "{:,.0f}", "R²":
                               use_container_width=True,
    )

    COLOR_MAP = {"Linear Regression": "#64748b", "XGBoost": "#22d3ee", "Attention

    c1, c2 = st.columns(2)
    with c1:
        fig = px.bar(comp_df, x="Model", y="MAPE (%)", color="Model", color_discr
                      title="Test-set MAPE by model (lower is better)")
        fig.update_layout(**PLOTLY_DARK, showlegend=False)
        st.plotly_chart(fig, use_container_width=True)
    with c2:
        fig2 = px.bar(comp_df, x="Model", y="R²", color="Model", color_discrete_m
                      title="Test-set R² by model (higher is better)")
        fig2.update_layout(**PLOTLY_DARK, showlegend=False)
        st.plotly_chart(fig2, use_container_width=True)

    st.info(f"📌 Best model on this test set (by MAPE): **{best_row['Model']}**")

    st.markdown(
        """Takeaway from Day 6 evaluation: XGBoost remained the strongest perfo
        raw accuracy for this tabular dataset. The attention-based spatial model
        beat it outright, but it adds explicit, interpretable neighbourhood reas

```



```

        "(which comps most influenced a price) that a plain gradient-boosted tree
        "natively provide – useful for explainability and as a foundation for fut
        "full graph-neural-network work."
    )

    fig_dir = "reports/figures"
    if os.path.isdir(fig_dir):
        imgs = [f for f in os.listdir(fig_dir) if f.endswith(".png")]
        if imgs:
            st.markdown("***Saved evaluation plots**")
            cols = st.columns(3)
            for i, img in enumerate(sorted(imgs)):
                cols[i % 3].image(os.path.join(fig_dir, img), caption=img, use_co

# -----
# TAB 3 – SPATIAL INSIGHTS
# -----

with tab3:
    st.subheader("Geographic Price Patterns")

    if sample_df is not None and {"lat", "long", "price"}.issubset(sample_df.colu
        fig_map = px.scatter_mapbox(
            sample_df, lat="lat", lon="long", color="price", size="price",
            color_continuous_scale="Turbo", zoom=9, height=520,
            hover_data=[c for c in ["price", "bedrooms", "bathrooms", "sqft_livin
            title="Sample listings colored by price",
        )
        fig_map.update_layout(mapbox_style="carto-darkmatter", margin=dict(l=0, r
        st.plotly_chart(fig_map, use_container_width=True)

    if "zipcode" in sample_df.columns:
        zip_price = (
            sample_df.groupby("zipcode")["price"].median().sort_values(ascend
        )
        st.markdown("***Top 10 zipcodes by median price**")
        fig_zip = go.Figure(go.Bar(
            x=zip_price.index.astype(str), y=zip_price.values,
            marker_color="#22d3ee"
        ))
        fig_zip.update_layout(**PLOTLY_DARK, height=350, xaxis_title="Zipcode
        st.plotly_chart(fig_zip, use_container_width=True)
    else:
        st.info(
            "Add `dataset/processed/sample_listings.csv` (with lat, long, price,
            "columns) to your repo to power the map and zipcode breakdown here."
        )

```

```

err_path = "dataset/zipcode_error_summary.csv"
if os.path.exists(err_path):
    err_df = pd.read_csv(err_path)
    st.markdown("***Average absolute % error by zipcode (from Day 6 evaluation  

    st.bar_chart(err_df.set_index(err_df.columns[0]))

# -----
# TAB 4 - ABOUT
# -----
with tab4:
    st.subheader("About This Project")
    st.markdown(
        """
**Project 3 - Construction & Real Estate: Geospatial Valuation via Spatial Embedd
Infotact Solutions & Co. - Advanced Data Science & Machine Learning Internship

**Dataset:** King County House Sales Dataset (Seattle, WA area)

**Pipeline:**
1. **Day 1 - EDA:** cleaning, outlier handling, price distribution, correlation a
2. **Day 2 - Feature Engineering:** Haversine distance to city center, house age,
    recency, structural ratios.
3. **Day 3 - Baseline Models:** Linear Regression and XGBoost, establishing the b
    MAPE/RMSE/R2 that the spatial model must beat.
4. **Day 4 - KNN Spatial Graph:** each house connected to its 10 nearest neighbou
    Haversine distance, forming the graph used by the spatial model.
5. **Day 5 - Attention-Based Spatial Model:** a lightweight PyTorch attention mec
    (GAT-style) where each house attends over its nearest neighbours, weighing mor
    informative "comps" more heavily.
6. **Day 6 - Evaluation:** side-by-side comparison of all three models, geographi
    error analysis, and discussion of trade-offs.

**Tech stack:** Python, Pandas, GeoPandas/Haversine, scikit-learn, XGBoost, PyTor
Plotly, Streamlit.
        """
    )
    st.caption("Author: Megha K A")

```